FinStructBench:

A Benchmark for Structured Information Retrieval
from Financial Documents Using Graph-Verifiable Questions

Agus Sudjianto

Abstract

We introduce **FinStructBench**, a benchmark for evaluating the ability of large language models (LLMs) to extract, aggregate, and reason over structured information in financial documents. Unlike existing QA benchmarks that rely on human-annotated answers, FinStructBench generates questions *automatically* from a document's underlying knowledge graph, making ground truth *provably correct by construction*. We define six question categories of increasing structural complexity—exact recall, threshold checking, cross-reference, counting, contradiction detection, and multi-hop reasoning—and provide five benchmark instances spanning major financial report types: model validation (SR 11-7), fair lending (ECOA/HMDA), stress testing (CCAR/DFAST), credit portfolio review (OCC), and Basel III capital adequacy (Pillar 3). Across 258 questions, a graph-based retrieval baseline achieves 100% accuracy by design, while Claude Sonnet 4 scores 45%, revealing a sharp *complexity gradient*: LLM accuracy drops from 86% on simple threshold queries to 0% on multi-hop reasoning. FinStructBench is released as an open-source toolkit enabling researchers to generate benchmark instances from any markdown-formatted financial document.

1 Introduction

Financial regulation demands precise information extraction from structured documents: capital ratios must be reported to basis-point accuracy, adverse impact ratios must be computed across protected classes, and stress-test projections must be traced across scenarios and time horizons. Errors in these tasks carry material consequences—regulatory findings, enforcement actions, and capital shortfalls.

Large language models are increasingly deployed for financial document analysis, yet existing benchmarks for financial NLP focus predominantly on sentiment analysis [Malo et al., 2014], named entity recognition [Alvarado et al., 2015], or free-form question answering [Chen et al., 2021, Zhu et al., 2021] where ground truth is established by human annotation. These benchmarks have three limitations in the structured-retrieval setting:

1. **Annotation cost and error.** Human annotators make mistakes on precise numeric lookups, especially in dense tables with hundreds of cells.
2. **Limited coverage.** Hand-crafted questions test only the patterns the annotator anticipated, missing systematic failure modes.
3. **No structural baseline.** Without a provably correct retrieval system, it is impossible to distinguish LLM errors from ambiguous ground truth.

We address all three limitations with **FinStructBench**, a benchmark construction methodology that:

- Ingests any markdown-formatted financial document into a *DocumentGraph* comprising exact numeric memory (ENM) entries and knowledge graph (KG) triples;
- Automatically generates questions by mining graph topology, with ground truth defined by graph traversal—making it provably correct;
- Provides a *graph-based retrieval baseline* (the graph itself) that achieves 100% by construction, against which any LLM or agentic system can be compared.

Our initial release includes 258 questions across five financial report types and six complexity categories. Results with Claude Sonnet 4 reveal a monotonic complexity gradient from 86% on threshold queries to 0% on multi-hop reasoning, demonstrating that current LLMs lack the structured traversal capabilities required for reliable financial document analysis.

2 Related Work

Financial NLP Benchmarks. FinQA [Chen et al., 2021] and TAT-QA [Zhu et al., 2021] evaluate numerical reasoning over financial tables, but rely on human-annotated answers and focus on arithmetic operations (addition, percentage change) rather than structural graph operations. ConvFinQA [Chen et al., 2022] extends to conversational settings but inherits the same annotation limitations. FLUE [Shah et al., 2022] aggregates multiple financial NLP tasks but does not address structured retrieval from regulatory documents.

Table QA. WikiTableQuestions [Pasupat & Liang, 2015] and HybridQA [Chen et al., 2020] benchmark table understanding but use general-domain tables without the precision requirements of financial regulation. SQA [Iyyer et al., 2017] addresses sequential table queries but does not test cross-table reasoning.

Document Understanding. DocVQA [Mathew et al., 2021] and financial variants evaluate visual document understanding, which is complementary to our text-based setting. Recent work on long-context evaluation [Li et al., 2024] tests retrieval from long documents but uses synthetic "needle-in-haystack" probes rather than domain-specific structural queries.

Knowledge Graph QA. KGQA benchmarks [Zhang et al., 2022] evaluate question answering over pre-existing knowledge graphs. FinStructBench differs in that the graph is *constructed from* the document, and the benchmark tests whether an LLM can implicitly perform the same traversal from the raw text.

3 Benchmark Construction

FinStructBench's construction pipeline has three stages: document ingestion, question generation, and evaluation. Each stage is fully automated and domain-agnostic.

3.1 Document Ingestion

Given a markdown-formatted financial document, the ingester produces a **DocumentGraph** $\mathcal{G} = (\mathcal{E}, \mathcal{T}, \Phi)$ with three components:

Exact Numeric Memory (ENM) $\mathcal{E}$: A key-value store mapping $(type, id) \to (value, hash)$ where each numeric value is stored with its SHA-256 hash for integrity verification. The *type* is derived from the sanitized section heading (e.g., `capital_adequacy_ratios`), and the *id* is a composite key encoding the entity, any secondary label columns, and (for multi-column tables) the column name, separated by `/`. For example, a stress test table with columns Scenario, Quarter, CET1, and Tier 1 produces keys such as (`capital_projections`, `Baseline/Q1 2026/CET1`), ensuring unambiguous lookup even when the same metric name appears in multiple rows. Each value is stored alongside its SHA-256 hash (computed over the IEEE-754 double representation) so that any silent corruption is detected at read time.

Knowledge Graph Triples $\mathcal{T}$: A set of $(head, relation, tail)$ triples extracted from table structure. For each row the ingester emits: (i) a `has_col` triple for every numeric cell, linking the entity to its string-formatted value; (ii) a categorical triple for every non-entity label column; (iii) a `passes` or `fails` triple if a boolean column is present; and (iv) an `in_section` triple linking the entity to its section tag.

Phase Encoders Φ: Inequality functions that map a metric type to a comparison operation. Each encoder stores the value range $[v_{\min}, v_{\max}]$ observed for that metric (used internally for geometric encoding) and exposes a `check_inequality`(v, τ, op) method supporting $\geq$, $>$, $\leq$, and $<$. Encoders are inferred from column names using regex matching against a dictionary of 11 financial metric types (Table 1).

Column Classification. The ingester classifies each column into one of four types using value-pattern analysis:

- **Numeric:** columns where >70% of non-empty values parse as floats (after stripping \$, %, commas, and parentheses for negative values).
- **Boolean:** columns where *every* non-empty value is one of: pass, fail, yes, no, true, false, weak, strong (case-insensitive).
- **Label:** non-numeric, non-boolean string columns.
- **Entity:** the primary identifier column, selected by name heuristic from a list of 16 common column names (e.g., "Feature", "Segment", "Scenario", "Product"), falling back to the first label column.

This classification is conservative by design: mixed columns (e.g., `3.2 (Pass)`) are classified as label rather than boolean or numeric, meaning the corresponding pass/fail relationship is not captured in the graph. Questions are generated only from data that *is* captured, so coverage may be incomplete but ground truth is never wrong.

Phase Encoders. Phase encoders serve two roles: (i) they define the value range for normalizing numeric values to $[0, 1]$, and (ii) they provide the threshold values and comparison operators used by the Threshold question generator. Table 1 lists the 11 encoder types, their column-name patterns, and the default threshold used for question generation (not the encoding range).

Algorithm 1 Document Ingestion Pipeline

Require: Markdown document D
Ensure: DocumentGraph $\mathcal{G} = (\mathcal{E}, \mathcal{T}, \Phi)$

Parse D into sections $\{s_1, \ldots, s_k\}$ by `##`/`###` heading hierarchy
Infer phase encoders Φ from column names across all tables
for each section s_i **do**
  for each markdown table T_j in s_i **do**
    Classify columns into *entity*, *numeric*, *boolean*, *label*
    Detect entity column by name heuristic; detect boolean column
    for each row r in T_j **do**
      $e \leftarrow$ entity value (or subsection name, or `row_`i)
      $e_{\text{full}} \leftarrow e \,/\, \text{label}_1 \,/\, \ldots \,/\, \text{label}_m$ {composite ID}
      for each numeric column c **do**
        $id \leftarrow e_{\text{full}}/c$ if multi-column, else e_{full}
        $\mathcal{E}[\text{section\_tag}, id] \leftarrow (v_c,\ \text{SHA-256}(v_c))$
        $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e, \texttt{has\_}c, v_c)\}$
      end for
      Add label triples: $(e, \texttt{has\_}\ell, v_\ell)$ for each label column ℓ
      if boolean column value $\in$ {true, yes, weak, fail} **then**
        $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e_{\text{full}}, \texttt{fails}, \text{tag})\}$
      else if $\in$ {false, no, strong, pass} **then**
        $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e_{\text{full}}, \texttt{passes}, \text{tag})\}$
      end if
    end for
  end for
  Extract bullet-list key-value pairs as metadata ENM entries and triples
end for
return $\mathcal{G}$

3.2 Question Generation

Questions are generated automatically by mining the DocumentGraph topology. Each generator implements a `generate(graph)` method that returns a pool of candidate questions, each comprising: (i) a natural-language question string, (ii) a deterministic graph-traversal function that computes the ground-truth answer, (iii) a structured LLM prompt with format instructions, and (iv) a typed answer format (float, int, bool, set, or string). We define six categories of increasing structural complexity:

1. **Exact Recall** — Look up a single ENM value by key. Iterates over all ENM entries and generates one question per entry.
   "What is the exact value of CET1 capital ratio?"
   Graph operation: $\mathcal{E}[\text{capital\_ratio}, \text{CET1}].\text{value}$
   Answer type: `float`. LLM is instructed to report the exact value with all decimal places.
2. **Threshold** — Check whether a value meets a domain-specific bound. For each ENM entry, the generator matches its category to a phase encoder via keyword lookup (e.g., a category containing "capital" maps to the `capital_ratio` encoder) and generates a boolean question for each applicable threshold.

Table 1: Phase encoder types: column-name patterns, default question thresholds, and comparison direction. Thresholds are domain-standard regulatory or statistical bounds; the Direction column shows whether values should be above ($\geq$) or below ($\leq$) the threshold.

Metric Type	Pattern	Threshold	Direction
AUC	`auc, auroc`	0.5	$\geq$
Accuracy	`accuracy, precision, recall, f1`	0.5	$\geq$
AIR	`air, impact_ratio`	0.8	$\geq$
Ratio	`ratio`	0.0	$\geq$
Rate	`rate`	0.05	$<$
Loss	`loss, charge.off`	0.5	$<$
Capital Ratio	`capital.*ratio, cet1, tier`	4.5	$\geq$
Importance	`importance, weight`	0.1	$\geq$
Divergence	`divergence, psi, kl`	0.3	$\geq$
P-value	`pvalue, p.value`	0.05	$<$
Percentage	`percentage, pct, share`	0.0	$\geq$

"Does the CET1 ratio meet the minimum capital requirement (≥4.5%)?"
Graph operation: $\phi_{\text{capital\_ratio}}(\mathcal{E}[\text{CET1}]) \geq 4.5$
Answer type: `bool`.

3. **Cross-Reference** — Find entities appearing in two relation types. Builds an index of relation $\rightarrow$ head entities, then generates one question for each pair of relations whose head-entity intersection has between 2 and 50 elements (filtering out trivial or degenerate overlaps).
"Which entities appear in both 'risk_rating' and 'has_npl'?"
Graph operation: $\{h : (h, r_1, \_) \in \mathcal{T}\} \cap \{h : (h, r_2, \_) \in \mathcal{T}\}$
Answer type: `set_str` (comma-separated list).

4. **Counting** — Aggregate over triple sets. Three sub-patterns: (a) count all triples with a given relation, (b) count entities with a specific (relation, tail) pair, and (c) count unique head entities per relation.
"How many entities have risk_rating = Substandard?"
Graph operation: $|\{h : (h, \text{risk\_rating}, \text{Substandard}) \in \mathcal{T}\}|$
Answer type: `int`.

5. **Contradiction** — Detect pass/fail inconsistencies across segments. Identifies entities that both pass and fail tests within the same feature group (matched by prefix on test-name strings). Generates three sub-types: (a) count of contradicting features, (b) set of contradicting features, and (c) per-feature boolean questions. Only applicable to instances with boolean columns (model validation in our release).
"Which features have both passing and failing segments?"
Graph operation: $\{f : \exists\,(\_, \texttt{passes}, f) \in \mathcal{T} \wedge \exists\,(\_, \texttt{fails}, f) \in \mathcal{T}\}$
Answer type: `int`, `set_str`, or `bool`.

6. **Multi-Hop** — Chained queries combining ENM lookups. Two sub-patterns: (a) *single-type*: find the argmin or argmax entry within one ENM type and report the entity name and value; (b) *cross-type*: find the argmin/max in type t_1, extract the base entity name (the first component of the composite key before `/`), then look up all entries for that entity in type t_2. Cross-type questions require at least 3 entries in t_1 and 2 in t_2, and are only generated when the base-entity match succeeds.
"Find the entity with the lowest loss rate, then look up its capital ratio."
Graph operation: $k^* = \arg\min_k \mathcal{E}[t_1, k]$; then $\{(k', v) : k' \in \mathcal{E}[t_2],\ \text{base}(k') = \text{base}(k^*)\}$
Answer type: `str` (multi-line formatted).

Sampling and Validation. Each generator produces a pool of all structurally valid candidates from the graph—the pool sizes range from 8 (multi-hop in Basel Capital, which has only 27 ENM entries) to 2,443 (cross-reference in Credit Portfolio, which has 2,334 triples). From each pool, up to N questions are sampled uniformly without replacement using a fixed random seed (default $N = 10$, seed 42) for reproducibility. Before inclusion, every candidate is *validated*: the graph answer function is re-executed and its output compared to the stored ground truth. Questions where the answer is `None`, an empty set, or inconsistent with the stored ground truth are discarded. This double-check guards against bugs in the answer function and ensures that the benchmark is self-consistent.

Ground Truth by Construction. A key property of FinStructBench is that ground truth is *defined* by graph traversal, not by human annotation. The ingester's correctness can be verified independently: we confirm that every stored ENM value appears verbatim in the source markdown (1,572/1,572 entries verified across all five instances). Given correct ingestion, the answer function is guaranteed correct by the determinism of key lookup, pattern matching, and set operations. This eliminates annotation noise and enables scaling to arbitrarily large question sets.

3.3 Evaluation Protocol

Each question is evaluated against two systems:

1. **Graph Baseline:** The answer function is executed directly on the DocumentGraph via deterministic traversal (key lookups, triple pattern matching, set operations). No learning or training is involved—the graph achieves 100% by construction and serves as the upper bound.
2. **LLM Under Test:** The LLM receives the full markdown document wrapped in `<report>` tags as context, along with a system prompt instructing it to answer precisely using only the provided data and to copy numbers exactly as they appear. Each question includes a format instruction specifying the expected answer tag (e.g., `ANSWER: 13.232`).

Response Parsing. LLM responses are parsed using type-specific parsers with two-stage extraction:

- **Float:** First attempts regex on `ANSWER:\s*<number>`; falls back to the last float literal in the response.
- **Int:** Same two-stage strategy; fallback extracts integers in the range $[1, 999]$.
- **Bool:** Matches `ANSWER:\s*(true|false|yes|no)`; falls back to keyword detection (`passes`, `does not`, `fails`).
- **Set:** Splits the `ANSWER:` line on commas and strips quotes.
- **String:** Returns the `ANSWER:` line verbatim.

Scoring. Scoring uses exact match with the following type-specific rules: float answers require $|a - g| < 10^{-6}$; integer answers require equality after casting; boolean answers require equality; set answers require exact set equality (superset answers are also accepted); string answers require case-insensitive equality. Tuple-valued answers (multi-hop) compare element-wise with float tolerance 10^{-3}.

Table 2: FinStructBench instance statistics.

Instance	ENM	Triples	Questions	Encoders	Regulatory Context
Model Validation	261	5,586	60	6	SR 11-7
Fair Lending	866	1,962	50	7	ECOA / HMDA
Stress Test	207	1,355	50	4	CCAR / DFAST
Credit Portfolio	211	2,334	50	5	OCC Guidelines
Basel Capital	27	785	48	5	Basel III Pillar 3
Total	**1,572**	**12,022**	**258**	—	—

4 Benchmark Instances

We release five benchmark instances covering major financial document types encountered in US banking regulation. Each instance is a synthetic but structurally realistic report: we first defined the table schemas, section structure, and entity names to mirror actual regulatory filings, then populated numeric cells with randomly generated values drawn from domain-appropriate distributions (e.g., capital ratios from $\mathcal{N}(12, 3)$ clipped to $[4, 25]$, approval rates from Beta$(5, 2)$). All instances are generated with fixed random seeds for reproducibility. Table 2 summarizes their characteristics.

Model Validation (SR 11-7). A model risk management report evaluating a credit scoring model across multiple segments. Contains accuracy metrics (AUC, precision, recall, F1), feature importance analysis (FANOVA), slicing performance by demographic segments, and resolution analysis. This is the only instance with contradiction questions, as it contains per-feature pass/fail assessments that can be inconsistent across segments.

Fair Lending (ECOA/HMDA). A fair lending analysis examining loan disposition patterns across protected classes (race, gender, age). Contains adverse impact ratios, pricing analysis with statistical significance tests, geographic distribution indicators, and model fairness metrics (demographic parity, equalized odds). The largest instance by ENM entries (866), reflecting the high dimensionality of cross-tabulated demographic data.

Stress Test (CCAR/DFAST). A capital adequacy stress test with macroeconomic scenarios (baseline, adverse, severely adverse) projected across nine quarters. Contains loan loss projections by portfolio segment, pre-provision net revenue forecasts, capital ratio trajectories, and capital action plans.

Credit Portfolio Review (OCC). A credit portfolio examination covering portfolio composition, credit quality metrics, risk rating distributions, concentration analysis, allowance adequacy, vintage analysis, migration matrices, and stress loss projections.

Basel Capital Adequacy (Pillar 3). A Basel III regulatory disclosure with capital composition, risk-weighted assets by exposure class, capital ratios, credit risk parameters (PD, LGD, EAD), market risk (VaR), operational risk (SMA), and leverage ratios. The smallest instance by ENM entries (27) but with complex hierarchical table structure.

Table 3: Overall benchmark results: Graph baseline vs. Claude Sonnet 4.

Instance	Questions	Graph	Claude Sonnet	Δ
Model Validation	60	60/60 (100%)	26/60 (43%)	−57 pp
Fair Lending	50	50/50 (100%)	30/50 (60%)	−40 pp
Stress Test	50	50/50 (100%)	21/50 (42%)	−58 pp
Credit Portfolio	50	50/50 (100%)	20/50 (40%)	−60 pp
Basel Capital	48	48/48 (100%)	18/48 (38%)	−63 pp
Total	**258**	**258 (100%)**	**115 (45%)**	**−55 pp**

Table 4: Results by question category (aggregated across all instances). Categories are ordered by decreasing LLM accuracy, revealing a monotonic complexity gradient.

Category	Graph Op.	Total	Graph	Claude	Claude %
Threshold	$\phi(v) \geq \tau$	50	50	43	86%
Cross-Reference	$S_1 \cap S_2$	50	50	31	62%
Exact Recall	$\mathcal{E}[k]$	50	50	22	44%
Counting	$\lvert\{h : (h, r, t)\}\rvert$	50	50	17	34%
Contradiction	$\exists\,\text{pass} \wedge \exists\,\text{fail}$	10	10	2	20%
Multi-Hop	$\text{chain}(f_1, f_2)$	48	48	0	0%
Total	—	**258**	**258**	**115**	**45%**

5 Experiments

5.1 Setup

We evaluate Claude Sonnet 4 (`claude-sonnet-4-20250514`) against the graph baseline across all five instances. The LLM receives the complete markdown document as context (500–1,000 lines) along with a structured prompt for each question specifying the expected answer format. We use 10 questions per category, sampled with seed 42.

5.2 Main Results

Table 3 presents the overall results. The graph baseline achieves 100% across all instances by construction. Claude Sonnet 4 achieves 45% overall, with per-instance scores ranging from 38% (Basel Capital) to 60% (Fair Lending).

5.3 Complexity Gradient

Table 4 reveals a striking complexity gradient. LLM accuracy decreases monotonically with the structural complexity of the required graph operation:

- **Threshold (86%):** Simple inequality checks against a single value. The LLM only needs to locate one number and compare it to a threshold—the simplest possible structured query.
- **Cross-Reference (62%):** Set intersection across two relation types. Requires identifying entities in two different contexts and computing their overlap.
- **Exact Recall (44%):** Precise numeric lookup. Despite being a single-hop operation, LLMs frequently return nearby values from the same table or confuse similarly-named entities.

Table 5: Claude Sonnet 4 accuracy by instance and category. Each cell shows correct/total. Shading indicates: ≥80%, 40–79%, <40%.

	Threshold	Cross-Ref	Exact Recall	Counting	Contradiction	Multi-Hop
Model Valid.	green!158/10	yellow!306/10	yellow!307/10	red!153/10	red!152/10	red!150/10
Fair Lending	green!1510/10	yellow!307/10	green!159/10	yellow!304/10	—	red!150/10
Stress Test	green!1510/10	green!158/10	red!150/10	red!153/10	—	red!150/10
Credit Port.	green!159/10	yellow!305/10	yellow!305/10	red!151/10	—	red!150/10
Basel Capital	yellow!306/10	yellow!305/10	red!151/10	yellow!306/10	—	red!150/8

- **Counting (34%):** Exhaustive enumeration and aggregation. Requires scanning all triples matching a pattern—LLMs tend to undercount or lose track in long tables.
- **Contradiction (20%):** Global consistency checking. Requires reasoning about whether a feature passes in some segments but fails in others—a form of negation-as-failure that LLMs struggle with.
- **Multi-Hop (0%):** Chained argmin/argmax followed by cross-type lookup. Claude Sonnet fails on all 48 multi-hop questions across all five instances, demonstrating a complete inability to perform sequential structured traversal.

5.4 Per-Instance Breakdown

Table 5 shows the full instance × category breakdown. Several patterns emerge:

Multi-hop is universally zero. Across all five instances, Claude Sonnet fails every multi-hop question (0/48). These questions require identifying an extremum (argmin/argmax) within one ENM type and then looking up the corresponding entity in a different ENM type—a two-step traversal that the LLM cannot reliably execute from raw text.

Instance difficulty varies. Fair Lending is the easiest instance (60%), likely because its tables have clear demographic category labels and the AIR threshold of 0.80 is well-known. Basel Capital is hardest (38%), with deeply nested tables and entities identified by composite keys (e.g., exposure class + sub-type + maturity band).

Exact recall difficulty depends on table structure. Stress Test exact recall scores 0/10 despite being a single-hop operation. This is because the stress test uses dense quarter × scenario matrices where multiple similar values appear in adjacent cells, causing the LLM to extract the wrong cell. Fair Lending scores 9/10 on exact recall because its tables have distinctive demographic labels.

5.5 Error Analysis

We analyze the 143 LLM failures across three representative error modes:

Adjacent Cell Confusion. In 37% of exact recall failures, the LLM returns a value from a neighboring cell in the same table. For example, when asked for "Q1 2026/Non-Interest Income," Claude returns the Non-Interest Expense value from the same row. This error is especially prevalent in the stress test instance, which uses wide matrices with 9 quarterly columns.

Partial Enumeration. In 68% of counting failures, the LLM provides a count that is lower than the ground truth. The model appears to enumerate entities until it reaches a "satisfying" count rather than exhaustively scanning all triples. For example, when asked to count entities with `risk_rating = Substandard`, Claude finds 4 of 7 entities, apparently stopping after processing the first table and missing occurrences in subsequent sections.

Multi-Hop Decomposition Failure. All 48 multi-hop failures follow the same pattern: the LLM either (a) answers only the first hop (reports the entity with the extreme value but not the cross-type lookup), (b) hallucinates the second-hop value, or (c) confuses which entity was the extremum. This suggests that current LLMs lack the ability to maintain intermediate results while executing a second query—a fundamental limitation for agentic financial analysis.

6 Discussion

Implications for Financial AI. The 55 percentage-point gap between graph-based retrieval and LLM performance has direct implications for the deployment of AI in financial regulation. Tasks that involve precise numeric extraction, exhaustive enumeration, or multi-step reasoning should not rely on LLM-only approaches. Hybrid architectures that combine LLM understanding with structured graph retrieval are likely necessary for production-grade financial document analysis.

Benchmark as a Development Tool. FinStructBench can serve as an evaluation harness during the development of retrieval-augmented generation (RAG) systems, agentic frameworks, and tool-use architectures for financial documents. The provably correct ground truth eliminates the need for human evaluation, enabling rapid iteration on system design.

Extensibility. The benchmark toolkit is designed for extensibility. New question categories can be added by implementing a generator class with a `generate(graph)` method. New instances can be created from any markdown document. The phase encoder system can be extended with domain-specific threshold rules.

Limitations. FinStructBench currently uses synthetic financial documents rather than real regulatory filings. While the synthetic reports are designed to be structurally realistic, real documents may contain additional complexity (e.g., footnotes, cross-references to external regulations, redacted sections). We plan to extend the benchmark with anonymized real-world instances in future work.

The benchmark evaluates a single LLM (Claude Sonnet 4) in a zero-shot, full-context setting. Future work should evaluate multiple models, few-shot prompting, chain-of-thought reasoning, and tool-augmented architectures.

7 Released Artifacts

FinStructBench is released as an open-source Python package with the following components:

1. **Toolkit** (`finstructbench/`): Document ingester, six question generators, scoring module, LLM caller, and benchmark orchestrator.
2. **Instances** (`instances/`): Five markdown financial reports with reproducible generator scripts.
3. **Results** (`results/`): Baseline results with per-question details in JSON format.
4. **CLI**: `python -m finstructbench run <doc.md>` with options for category limit, model, and graph-only mode.

8 Conclusion

We presented FinStructBench, a benchmark for structured information retrieval from financial documents with provably correct ground truth. Our results reveal a sharp complexity gradient in LLM performance: Claude Sonnet 4 achieves 86% on simple threshold checks but 0% on multi-hop reasoning, with an overall score of 45% against the 100% graph baseline. These findings demonstrate that current LLMs are insufficient for reliable financial document analysis and motivate the development of hybrid graph-augmented architectures. FinStructBench provides a rigorous, extensible evaluation framework for measuring progress toward this goal.

References

Alvarado, J. C. S., Verspoor, K., and Baldwin, T. (2015). Domain adaption of named entity recognition to support credit risk assessment. In *Proceedings of the Australasian Language Technology Association Workshop*, pp. 84–90.

Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., and Wang, W. Y. (2020). HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of EMNLP*, pp. 1026–1036.

Chen, Z., Chen, W., Smiley, C., Shah, S., Borber, I., Leu, S., Zhu, Y., Kumar, S., Zhang, J., and Wang, W. Y. (2021). FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*, pp. 7862–7872.

Chen, Z., Li, S., Smiley, C., Ma, Z., Shah, S., and Wang, W. Y. (2022). ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of EMNLP*, pp. 6279–6292.

Iyyer, M., Yih, W., and Chang, M. (2017). Search-based neural structured learning for sequential question answering. In *Proceedings of ACL*, pp. 1821–1831.

Li, Y., Packer, C., and Gonzalez, J. E. (2024). How long can open-source LLMs truly promise on context length? *arXiv preprint arXiv:2402.17688*.

Malo, P., Sinha, A., Korhonen, P., Wallenius, J., and Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4):782–796.

Mathew, M., Karatzas, D., and Jawahar, C. V. (2021). DocVQA: A dataset for VQA on document images. In *Proceedings of WACV*, pp. 2200–2209.

Pasupat, P. and Liang, P. (2015). Compositional semantic parsing on semi-structured tables. In *Proceedings of ACL*, pp. 1470–1480.

Shah, R., Chawla, K., Eidnani, D., Shah, A., Du, W., Chava, S., Raman, N., Smiley, C., Chen, J., and Yang, D. (2022). When FLUE meets FLANG: Benchmarks and large pre-trained language model for financial domain. In *Proceedings of EMNLP*, pp. 2322–2335.

Zhang, J., Chen, B., Zhang, L., Ke, X., and Ding, H. (2022). Neural, symbolic and neural-symbolic reasoning on knowledge graphs. *AI Open*, 3:109–124.

Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., and Chua, T. (2021). TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of ACL*, pp. 3277–3287.